

ES6+ features

COMPLETE LEARNING GUIDE

What You'll Learn

ECMAScript 2015 (ES6) and the yearly releases since introduced syntax that fundamentally changed how idiomatic JavaScript is written: destructuring, arrow functions, template literals, spread/rest, default parameters, real classes, native modules, and — more recently — optional chaining and nullish coalescing. None of these are optional extras in a modern codebase; they're the baseline syntax every React/Vue/Angular codebase and every modern job assumes you already read fluently. This submodule covers each feature with practical, realistic examples rather than toy syntax demos, and pays particular attention to the subtle gotchas — shallow vs. deep copying with spread, default vs. named exports, and when ?? is correct where || silently isn't.

1. Destructuring: Arrays and Objects

Destructuring pulls values out of arrays or objects directly into named variables in a single expression, replacing repetitive `const x = obj.x; const y = obj.y;` boilerplate. Object destructuring matches by property name (order doesn't matter) and supports renaming and default values; array destructuring matches by position (order matters) and is how multiple return values are typically 'unpacked' from a function or a `Promise.all` result. Destructuring also works directly in function parameters, which is the standard way React components accept props.

JAVASCRIPT

```
const user = { name: 'Aisha', role: 'admin', email: 'aisha@example.com' };
const { name, role: userRole, phone = 'N/A' } = user;
// name = 'Aisha', userRole = 'admin', phone = 'N/A' (default, since not present)

const [first, second, ...rest] = [10, 20, 30, 40];
// first = 10, second = 20, rest = [30, 40]

function UserCard({ name, email }) { // destructuring directly in the parameter
  list
  return `${name} <${email}>`;
}
```

2. Template Literals

Template literals (backtick strings) support `${}` interpolation of any expression directly inside the string, and natively support multi-line strings without `\n` concatenation. Tagged templates (an advanced form where a function processes the literal's pieces) power libraries like styled-components, but the everyday use — clean interpolation replacing `'...' + var + '...'` concatenation — is what you'll reach for constantly.

JAVASCRIPT

```
const name = 'Rohan';
const itemCount = 3;
const message = `Hi ${name}, you have ${itemCount} item${itemCount !== 1 ? 's' : ''} in your cart.`;
```

3. Spread and Rest Operators

The `...` syntax means two different things depending on context. As spread, it expands an array or object's elements/properties into a new array/object literal or into function arguments — the standard way to copy or merge arrays/objects immutably, and to build a new state object in React without mutating the original. As rest, appearing in a destructuring pattern or function parameter list, it collects the remaining elements/arguments into a single array. Crucially, spread performs a shallow copy — nested objects/arrays inside the spread result still point to the same nested references as the original, so mutating a nested property affects both copies.

JAVASCRIPT

```
const original = { name: 'Aisha', address: { city: 'Bangalore' } };
const copy = { ...original, name: 'Meera' }; // shallow copy, name overridden

copy.address.city = 'Pune';
console.log(original.address.city); // 'Pune' — nested object was shared, not deep-copied!

function sum(...numbers) { // rest — collects all arguments into an array
  return numbers.reduce((total, n) => total + n, 0);
}
sum(1, 2, 3); // 6
```

4. Default Parameters

Function parameters can declare a default value used only when the caller passes undefined for that argument (explicitly passing null does not trigger the default). Defaults are evaluated left-to-right at call time and can reference earlier parameters, which is useful for parameters that depend on each other.

JAVASCRIPT

```
function createUser(name, role = 'member', id = crypto.randomUUID()) {  
  return { id, name, role };  
}  
createUser('Aisha'); // role defaults to 'member', id auto-generated
```

5. ES Modules: import/export

Native JavaScript modules replace older patterns (global script tags, IIFEs, CommonJS require in browser code) with explicit, statically analyzable dependencies declared via import/export. A module can have any number of named exports (export const x = ...) imported with matching names in {}, and at most one default export (export default ...) imported under any chosen name without braces. Because imports/exports are static (resolved before code runs, not computed at runtime), bundlers can perform tree-shaking — eliminating exported code that's never actually imported anywhere, which shrinks the final bundle sent to the browser.

JAVASCRIPT

```
// mathUtils.js  
export const PI = 3.14159;  
export function double(n) { return n * 2; }  
export default function square(n) { return n * n; }  
  
// app.js  
import square, { PI, double } from './mathUtils.js';
```

6. Classes and Inheritance

ES6 classes are primarily syntax over JavaScript's existing prototype-based inheritance, not a fundamentally new object model. class declares a constructor and methods (which live on the prototype, shared across instances, unlike arrow-function class fields which are per-instance); extends and super() set up inheritance and let a subclass call its parent's constructor/methods. Modern class syntax also supports true private fields (prefixed with #), inaccessible from outside the class, unlike the older convention of prefixing a property with an underscore, which was private by convention only.

JAVASCRIPT

```
class Shape {  
  #id = crypto.randomUUID(); // truly private – inaccessible outside the class  
  constructor(name) { this.name = name; }  
  describe() { return `${this.name} (${this.#id})`; }  
}  
  
class Circle extends Shape {  
  constructor(radius) {  
    super('Circle'); // must call super() before using `this` in a subclass  
    this.radius = radius;  
  }  
  area() { return Math.PI * this.radius ** 2; }  
}
```

7. Optional Chaining and Nullish Coalescing

Optional chaining (?.) short-circuits to undefined instead of throwing when accessing a property on null/undefined, replacing verbose manual guards like `obj && obj.a && obj.a.b`. It works on property access, method calls (`obj.method?.()`), and array indexing (`arr?.[0]`). Nullish coalescing (??) provides a fallback value only when the left side is null or undefined — critically different from `||`, which falls back on any falsy value, including 0, "", and false, all of which are often legitimate, intentional values that shouldn't be silently replaced.

JAVASCRIPT

```
const user = { profile: { settings: null } };
const theme = user.profile?.settings?.theme ?? 'light'; // undefined chain ->
fallback 'light'

function getDiscount(percent) {
  // BUG with ||: a real, valid 0% discount would incorrectly become the default
  const withOr = percent || 10;
  // Correct with ??: only null/undefined trigger the default, 0 is respected
  const withNullish = percent ?? 10;
  return withNullish;
}
getDiscount(0); // 0, correctly
```

Common Pitfalls

- Assuming spread (`{...obj}`) performs a deep copy — it's shallow, so nested objects/arrays remain shared references.
- Using `||` for a default value on a parameter where 0, "", or false are valid inputs, incorrectly overriding them — use `??` instead.
- Destructuring a property that doesn't exist yet on a possibly-null object without optional chaining, throwing a `TypeError`.
- Mixing up default and named exports, causing 'X is not a function' or unexpected undefined imports.
- Forgetting that rest parameters must be the last parameter in a function's parameter list.
- Using arrow functions as object/class methods when you need `this`` to refer to the instance — arrow functions capture the enclosing lexical `this`` instead.
- Forgetting to call `super()` before using `this`` in a subclass constructor, which throws a `ReferenceError`.

Key Takeaways

- Destructuring with defaults and renaming eliminates most manual property-access boilerplate — and is how React components typically read props.
- Spread (`...`) is shallow — nested structures still share references after a spread copy.

- ?? (nullish coalescing) is the correct default-value operator whenever 0, "", or false are legitimate values; || is not.
- ES module imports/exports are statically analyzable, which is what enables bundlers to tree-shake unused code out of the final bundle.
- Class fields with # are genuinely private at the language level, unlike the older underscore-prefix convention.

Learn More

MDN — Destructuring assignment

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring_assignment

MDN — Optional chaining (?.)

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Optional_chaining

javascript.info — Modules

<https://javascript.info/modules>